MINISTRY OF EDUCATION, CULTURE AND RESEARCH
OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Report

on Laboratory Work no. 4
Operating Systems
Theme: The xv6 Shell from the Outside

Done by: st. gr. FAF-242, **Moroșanu Ion**
Checked by: **Reițman Patricia**, *university assistant*
FCIM, UTM

Chișinău – 2026

TABLE OF CONTENTS

INTRODUCTION **3**
Purpose of the work 3
1 FIXING SLEEP FOR THE CURRENT XV6 ..... **4**
2 BASH VERSUS THE XV6 SHELL **6**
2.1 The experiment 6
2.2 ls after cd d 7
2.3 Answers to the questions 9
3 WHO EXPANDS *? A PROGRAM THAT PRINTS ITS ARGV **10**
3.1 The program args 10
3.2 The results 10
3.3 Answers to the questions 12
4 READING THE SHELL **13**
4.1 The kinds of command 13
4.2 The handling of cd 14
4.3 The characters the shell understands 14
4.4 Answers to the questions 15
CONCLUSIONS **16**
REFERENCES **17**

INTRODUCTION

Purpose of the work

The purpose of the fourth laboratory work is to study the shell of xv6 from the outside. The whole user interface of xv6 is one program, user/sh.c, about 500 lines long [1]. In this work it was used side by side with bash to find out by experiment what every shell must do and what bash adds on top of that. A small program that prints its arguments was written for both systems to show exactly who expands the pattern * and who removes the quotes. Finally, the source of the xv6 shell was read to explain the results.

The following tasks were set for the work:

a) to make the program sleep from the second laboratory work build and run on the current xv6;

b) to run the same ten command lines in bash and in the xv6 shell and to compare the results;

c) to write the program args for xv6 and for Linux and to run it with patterns, quotes and variables in both shells;

d) to read user/sh.c and to find the kinds of command it knows, the place where cd is handled and the code for * and $.

The work was done on the same Ubuntu 26.04 virtual machine as the previous laboratory works. xv6 was already installed and built in the second laboratory work in ~/xv6-riscv, with the RISC-V cross compiler riscv64-linux-gnu-gcc and the emulator QEMU, so the setup script of the course was not needed. xv6 is started with `make qemu` and stopped with Ctrl-A followed by X.

1 FIXING SLEEP FOR THE CURRENT XV6

In August 2025 the authors of xv6 renamed the system call sleep() to pause() [3]. The command `grep -n "pause\|sleep" user/user.h` searches the list of system calls that user programs can call for both names; the option -n adds the line numbers and \| means OR. The only match is line 25, `int pause(int);`, so this tree uses the new name (Figure 1.1).

```
wancea@OS-Lab1-2:~$ cd ~/xv6-riscv
wancea@OS-Lab1-2:~/xv6-riscv$ grep -n "pause\|sleep" user/user.h
25:int pause(int);
wancea@OS-Lab1-2:~/xv6-riscv$
```

Figure 1.1 – The grep result in user.h

The program user/sleep.c from the second laboratory work already calls pause() and therefore builds without changes (Figure 1.2). It checks that it received exactly one argument besides its own name, and otherwise prints a usage message to the file descriptor 2, the standard error, and ends with the exit code 1. The function atoi converts the text of the argument into a number, and pause asks the kernel to suspend the process for that many clock ticks. The line `$U/_sleep\` is present in the list UPROGS of the Makefile (line 135), so the program is compiled and copied onto the disk image of xv6. The build gives the file the name _sleep, and the program mkfs, which builds the disk image, removes the underscore, so inside xv6 the program is called sleep.

```
wancea@OS-Lab1-2:~/xv6-riscv$ cat user/sleep.c
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  pause(atoi(argv[1]));   // system call into the kernel
  exit(0);
}
wancea@OS-Lab1-2:~/xv6-riscv$ grep -n "_sleep" Makefile
135:    $U/_sleep\
wancea@OS-Lab1-2:~/xv6-riscv$
```

Figure 1.2 – The updated sleep.c and its line in UPROGS

After `make qemu` the command `sleep 20` paused the shell and the prompt returned after a few seconds, and `sleep` without an argument printed `usage: sleep <ticks>` (Figure 1.3). A tick of the xv6 timer is about a tenth of a second in QEMU; inside this virtual machine the emulation is slower, so the pause lasted a little longer than two seconds.



Figure 1.3 – sleep running in xv6, with and without an argument

2 BASH VERSUS THE XV6 SHELL

2.1 The experiment

The same ten command lines were run first in bash, in an empty test folder ~/shtest that contains a file named README (Figure 2.1), and then at the $ prompt of xv6, whose disk also contains a README file in the root folder (Figures 2.2 to 2.4). Table 2.1 shows what each shell printed and why.

```
wancea@OS-Lab1-2:~/os-lab3/play$ mkdir -p ~/shtest && cd ~/shtest && rm -rf f d &&
touch README && ls
README
wancea@OS-Lab1-2:~/shtest$ echo hello > f ; cat < f
hello
wancea@OS-Lab1-2:~/shtest$ ls | grep READ
README
wancea@OS-Lab1-2:~/shtest$ (echo a; echo b) | wc
      2       2       4
wancea@OS-Lab1-2:~/shtest$ echo one ; echo two
one
two
wancea@OS-Lab1-2:~/shtest$ echo bg &
[1] 4300
bg
[1]+  Done                    echo bg
wancea@OS-Lab1-2:~/shtest$ ls *
README  f
wancea@OS-Lab1-2:~/shtest$ echo $HOME
/home/wancea
wancea@OS-Lab1-2:~/shtest$ echo "a b"
a b
wancea@OS-Lab1-2:~/shtest$ mkdir d ; cd d ; ls ..
README  d  f
wancea@OS-Lab1-2:~/shtest/d$
```

Figure 2.1 – The commands run in bash

```
$ echo hello > f ; cat < f
hello
$ ls | grep READ
README          2 2 2441
$ (echo a; echo b) | wc
2 2 4
$ echo one ; echo two
one
two
$ echo bg &
bg$

$ ls *
ls: cannot open *
$ echo $HOME
$HOME
$ echo "a b"
"a b"
$
```

Figure 2.2 – The same commands run in xv6

Table 2.1 – The same command lines in bash and in xv6

Command	Tests	bash	xv6	Why
`echo hello > f ; cat < f`	redirection	hello	hello	Both shells implement < and >.
`ls | grep READ`	pipe	README	README 2 2 2441	Both connect the programs with a pipe; the ls of xv6 also prints the type, the inode number and the size.
`(echo a; echo b) | wc`	grouping, list	2 2 4	2 2 4	Both support parentheses and the list operator ;.
`echo one ; echo two`	sequence	one, two	one, two	Both run the commands one after the other.
`echo bg &`	background	[1] 4300, bg, then Done	bg, and the prompt at once	Both run the command in the background; only bash numbers the job and reports when it ends.
`ls *`	globbing	README f	ls: cannot open *	xv6 does not expand *, so ls receives it literally.
`echo $HOME`	variables	/home/wancea	$HOME	xv6 has no shell variables.
`echo "a b"`	quotes	a b	"a b"	xv6 does not remove quotes; they are passed to echo.
`mkdir d ; cd d ; ls ..`	PATH	README d f	exec cd failed, then the listing of /	xv6 treats cd as a builtin only at the start of a line, so here it tried to run a program named cd; ls .. then ran in /, whose parent is / itself.
arrow up, Tab, Ctrl+R	history, completion	previous command, completion, history search	raw control codes, then exec failed	xv6 has no line editor; the keys are sent as characters.

2.2 ls after cd d

To test the search for programs, the folder d was entered with `cd d` alone on the line, which xv6 handles as a builtin. After that the command `ls` failed with `exec ls failed`, while `/ls` and `../ls` worked and listed the two entries . and .. of the new folder (Figure 2.3).

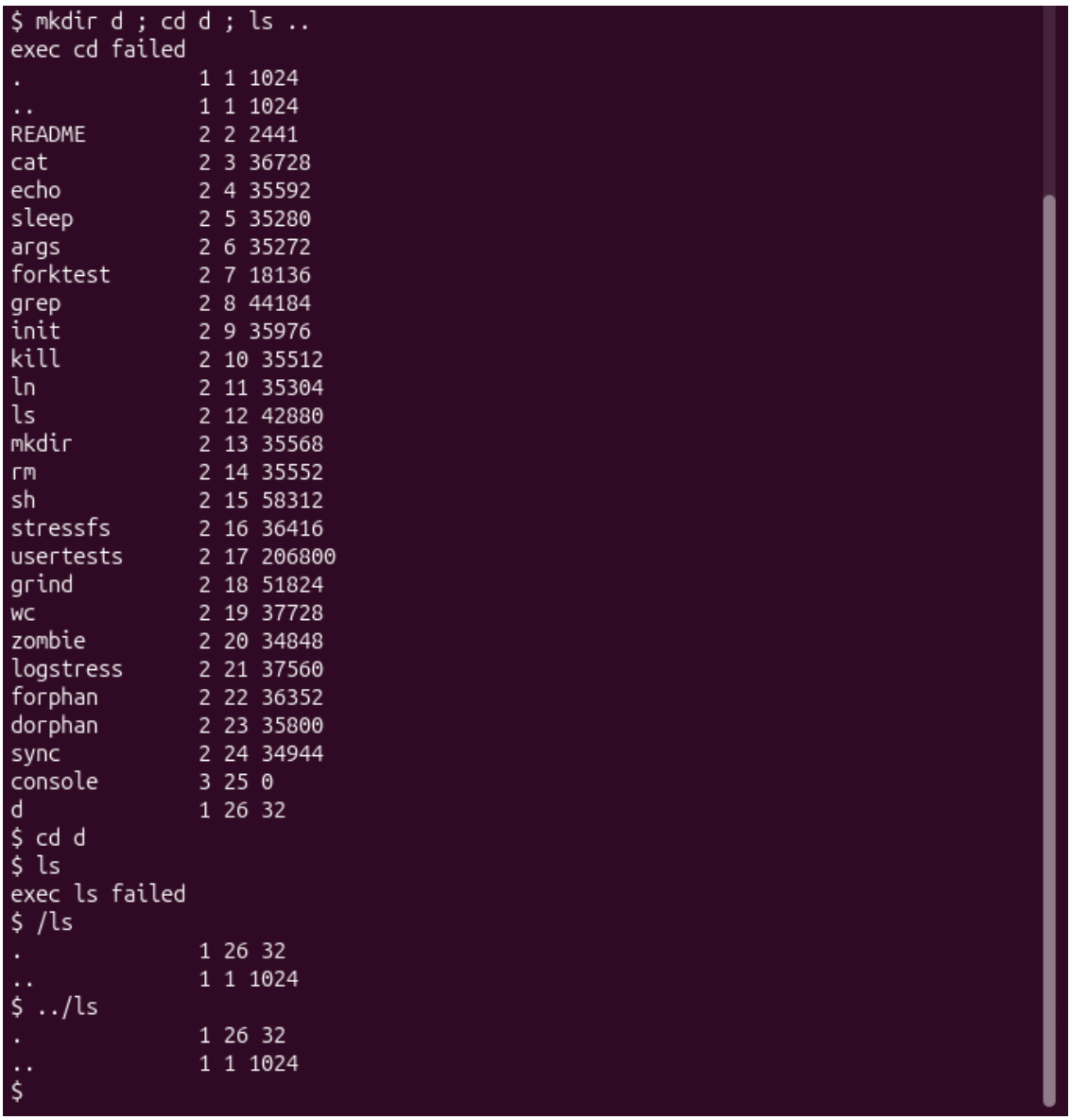


Figure 2.3 – ls failing after cd d, and the working alternatives

Figure 2.4 shows the keys for history and completion. Before Enter (upper part) the arrow key had already moved the cursor up one line, because xv6 echoes its escape sequence ESC [A back to the terminal unchanged, and the terminal of Linux interprets it as "cursor up"; Tab moved the cursor to the next tab stop and Ctrl+R printed nothing visible. After Enter (lower part) the shell tried to run a program whose name consists of these control characters, and its error message `exec ... failed`, which contains the same escape sequence, was printed partly over the previous line, so the screen shows `cd  failed` and `ec`.



Figure 2.4 – Arrow up, Tab and Ctrl+R in xv6, before and after Enter

2.3 Answers to the questions

1. Both shells have output and input redirection (<, > and also >>), pipes, grouping with parentheses, the list operator ; and background execution with &. Only bash has pattern expansion (globbing), variables, quote removal, the search of programs through PATH, cd anywhere on a line, job numbers and job reports, the command history, line editing and Tab completion.

2. The xv6 shell calls exec with the name exactly as it was typed. There is no PATH, so a name without a slash is looked up relative to the current folder. All xv6 programs are in the root folder /, so in / the name ls means /ls, but after `cd d` it means /d/ls, which does not exist. The program can still be run by its path: `/ls` (absolute) or `../ls` (relative).

3. In daily work the missing feature that would be missed most is the command history with the arrow keys, closely followed by globbing. The easiest one to add is a simple PATH: when exec fails for a name without a slash, the shell can try again with "/" in front of the name. That is two or three lines in the EXEC case of runcmd, while history needs a whole line editor.

3 WHO EXPANDS *? A PROGRAM THAT PRINTS ITS ARGV

3.1 The program args

The program user/args.c prints the number of its arguments argc and then every argument argv[i] between square brackets, so that the exact start and end of each argument, including spaces inside it, are visible. It was added to UPROGS with the line `$U/_args\` after the line of sleep (Figure 3.1). After `make qemu` the build compiled args.c with the RISC-V cross compiler and mkfs copied the program onto the disk image.

```
wancea@OS-Lab1-2:~/xv6-riscv$ cat user/args.c ; grep -n -B1 -A1 "_args" Makefile ;
grep -c "_args" Makefile
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  printf("argc = %d\n", argc);
  for(int i = 0; i < argc; i++)
    printf("argv[%d] = [%s]\n", i, argv[i]);
  exit(0);
}
135-    $U/_sleep\
136:    $U/_args\
137-    $U/_forktest\
1
wancea@OS-Lab1-2:~/xv6-riscv$
```

Figure 3.1 – args.c and the UPROGS line

The Linux version args_linux.c is the same program with `#include <stdio.h>` instead of the two xv6 headers and `return 0` instead of `exit(0)`. An xv6 program must call exit, because there is no C runtime that ends the process when main returns, while on Linux the runtime does this. It was compiled with `gcc -o args args_linux.c` in the folder ~/os-lab3/play, which contains the files a.c and b.c.

3.2 The results

The same four lines were run in both shells (Figures 3.2 and 3.3). In bash the program was started as ./args, because the current folder is not in PATH.

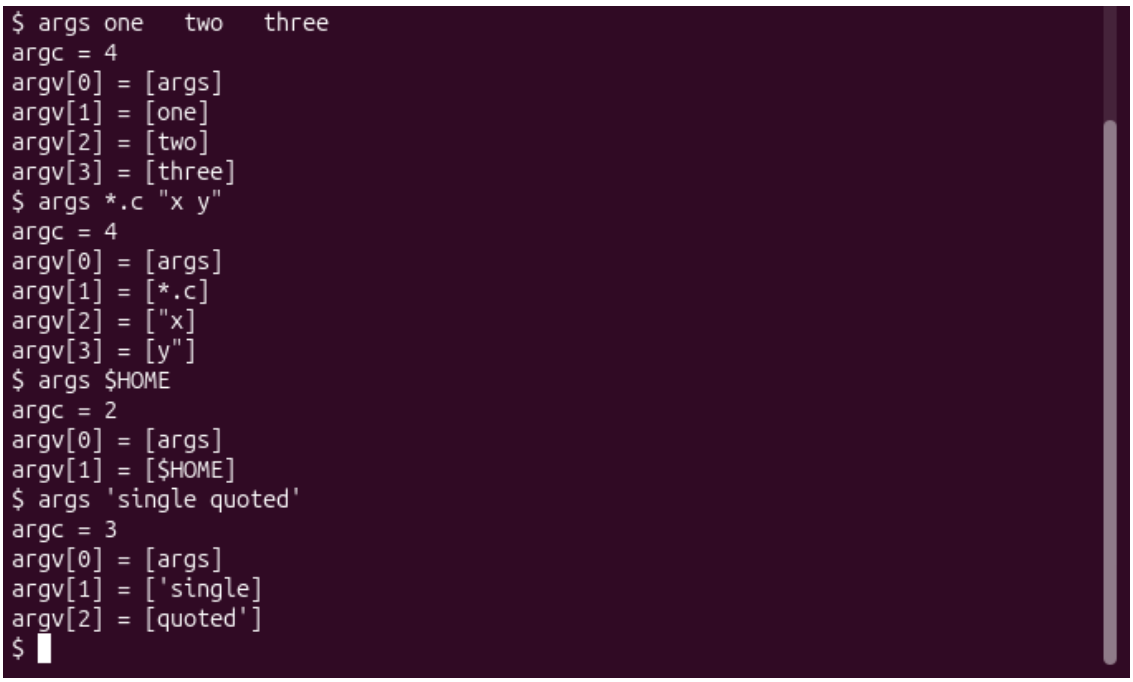


Figure 3.2 – args output in xv6

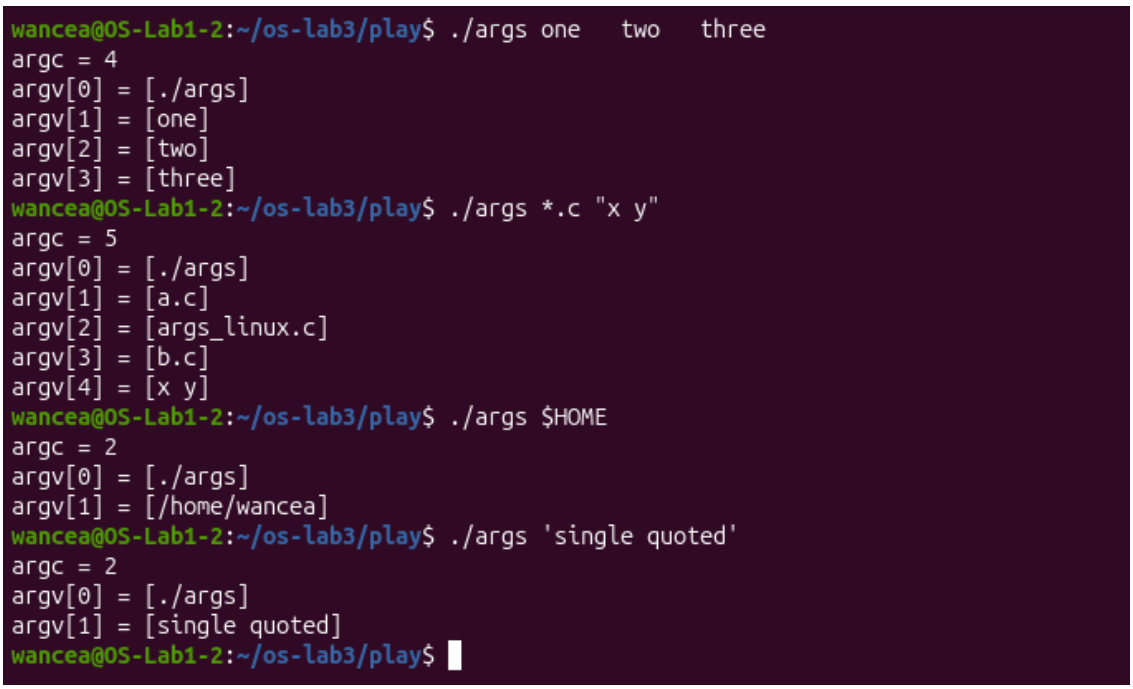


Figure 3.3 – args_linux output in bash

Table 3.1 – The arguments received by the program

Command line	xv6	bash
`args one two three`	argc = 4: args, one, two, three	argc = 4: ./args, one, two, three
`args *.c "x y"`	argc = 4: args, *.c, "x, y"	argc = 5: ./args, a.c, args_linux.c, b.c, x y
`args $HOME`	argc = 2: args, $HOME	argc = 2: ./args, /home/wancea
`args 'single quoted'`	argc = 3: args, 'single, quoted'	argc = 2: ./args, single quoted

3.3 Answers to the questions

1. For `args *.c "x y"` the xv6 program received 4 arguments: args, *.c, "x and y". The quotes stayed inside the arguments and the space between x and y split them. The Linux program received 5 arguments: ./args, a.c, args_linux.c, b.c and x y, without quotes and with the space kept inside one argument.

2. The shell turns *.c into file names and removes the quotes. The C code of the program is the same in both systems and only prints what it receives, and the kernel call exec only copies the list of strings that it is given into the new process. The only difference between the two runs is the shell, and only bash rewrites the command line before it starts the program.

3. The extra spaces in `one  two  three` disappeared in both shells. A shell splits the line at any number of spaces and keeps only the words, so the program received three separate arguments with no spaces.

4 READING THE SHELL

4.1 The kinds of command

The file user/sh.c has 499 lines (Figure 4.1). The parser turns a command line into a tree of commands, and `grep -n "#define" user/sh.c` shows the five kinds of node in lines 8 to 12: EXEC 1, REDIR 2, PIPE 3, LIST 4 and BACK 5. Line 14, MAXARGS 10, is not a kind of command but the maximum number of arguments of one command.

```
wancea@OS-Lab1-2:~/xv6-riscv$ wc -l user/sh.c
499 user/sh.c
wancea@OS-Lab1-2:~/xv6-riscv$ grep -n "#define" user/sh.c
8:#define EXEC  1
9:#define REDIR 2
10:#define PIPE  3
11:#define LIST  4
12:#define BACK  5
14:#define MAXARGS 10
```

Figure 4.1 – The command kinds defined in sh.c

The command `grep -n "case " user/sh.c` finds four groups of case lines (Figure 4.2). Lines 75 to 125 are the switch of the function runcmd, where each kind of command is executed: EXEC calls exec, REDIR closes a file descriptor and opens the file in its place, LIST runs the left command, waits for it and then runs the right one, PIPE creates a pipe and two child processes, and BACK forks a child and does not wait for it. Lines 284 to 294 are in gettoken, which recognises the special characters, lines 391 to 397 are in parseredirs, which builds the redirections for <, > and >> (the token +), and lines 469 to 493 are in nulterminate, which ends the strings of the parsed tree.

```
wancea@OS-Lab1-2:~/xv6-riscv$ grep -n "case " user/sh.c
75:  case EXEC:
83:  case REDIR:
93:  case LIST:
101:  case PIPE:
125:  case BACK:
284:  case 0:
286:  case '|':
287:  case '(':
288:  case ')':
289:  case ';':
290:  case '&':
291:  case '<':
294:  case '>':
391:    case '<':
394:    case '>':
397:    case '+': // >>
469:  case EXEC:
475:  case REDIR:
481:  case PIPE:
487:  case LIST:
493:  case BACK:
wancea@OS-Lab1-2:~/xv6-riscv$
```

Figure 4.2 – Where each kind is run

4.2 The handling of cd

The main loop of the shell is in the function main, lines 145 to 178 (Figure 4.3). It reads a line with getcmd, skips the leading spaces and checks in line 166 whether the line starts with the characters c, d and a space. In that case the shell itself calls chdir in line 169 and prints `cannot cd` if the call fails. For every other line it calls fork1 and the child runs the parsed command with runcmd, while the parent waits for it with wait. The comment in line 167 states the reason: "Chdir must be called by the parent, not the child."

```
145  int
146  main(void)
147  {
148    static char buf[100];
149    int fd;
150
151    // Ensure that three file descriptors are open.
152    while ((fd = open("console", O_RDWR)) >= 0) {
153      if (fd >= 3) {
154        close(fd);
155        break;
156      }
157    }
158
159    // Read and run input commands.
160    while (getcmd(buf, sizeof(buf)) >= 0) {
161      char *cmd = buf;
162      while (*cmd == ' ' || *cmd == '\t')
163        cmd++;
164      if (*cmd == '\n') // is a blank command
165        continue;
166      if (cmd[0] == 'c' && cmd[1] == 'd' && cmd[2] == ' ') {
167        // Chdir must be called by the parent, not the child.
168        cmd[strlen(cmd) - 1] = 0; // chop \n
169        if (chdir(cmd + 3) < 0)
170          fprintf(2, "cannot cd %s\n", cmd + 3);
171      } else {
172        if (fork1() == 0)
173          runcmd(parsecmd(cmd));
174        wait(0);
175      }
176    }
177    exit(0);
178  }
```

Figure 4.3 – The code that handles cd

4.3 The characters the shell understands

The string symbols in line 269 lists every character that the xv6 shell treats as special: < | > & ; ( ). The search for character constants '*' or '$' in sh.c and the search for PATH, getenv or glob found nothing and ended with the exit code 1 (Figure 4.4).

```
wancea@OS-Lab1-2:~/xv6-riscv$ grep -n "symbols\|whitespace" user/sh.c | head -4
268:char whitespace[] = " \t\r\n\v";
269:char symbols[] = "<|>&;()";
278:  while (s < es && strchr(whitespace, *s))
303:    while (s < es && !strchr(whitespace, *s) && !strchr(symbols, *s))
wancea@OS-Lab1-2:~/xv6-riscv$ grep -n "'[*$]'" user/sh.c ; echo "grep exit code: $?
"
grep exit code: 1
wancea@OS-Lab1-2:~/xv6-riscv$ grep -n "PATH\|getenv\|glob" user/sh.c ; echo "grep e
xit code: $?"
grep exit code: 1
wancea@OS-Lab1-2:~/xv6-riscv$
```

Figure 4.4 – The special characters of sh.c and the search for * and $

4.4 Answers to the questions

1. The five kinds of command are EXEC, a plain command, which needs no special character; REDIR, produced by <, > or >> (in the code >> is the token +); PIPE, produced by |; LIST, produced by ;; and BACK, produced by &. Parentheses do not create a kind of their own: the parser returns the command inside them.

2. cd is handled in the function main, lines 166 to 170, before the shell forks. It cannot be a separate program, because a program runs in a child process and chdir changes only the current folder of the process that calls it. A cd program would change its own folder and exit, and the shell would stay where it was. The same check also explains Table 2.1: cd works only at the start of a line, so in `mkdir d ; cd d ; ls ..` it was sent to runcmd as an ordinary program and exec failed.

3. sh.c contains no code that handles * or $, and its list of special characters has neither of them, nor any quotes. The xv6 shell only splits the line at spaces and at the characters < | > & ; ( ), so *, $HOME and quotes reach the programs unchanged. This explains the results of Part 2 for globbing, variables and quotes and the arguments received by args in Part 3.

CONCLUSIONS

The work showed what a shell must do and what is optional. The core of every shell is a short loop: read a line, split it into words and special characters, create a child process with fork, replace it with the program through exec and wait for it to end. On top of that loop the xv6 shell implements redirection, pipes, lists, grouping and background jobs in fewer than 500 lines, and the results of these features in xv6 were the same as in bash.

Everything else that bash does is extra work done on the command line before the program starts: expanding patterns, replacing variables, removing quotes, searching PATH and editing the line with history and completion. The program args proved this directly. The same C code received `*.c`, `"x` and `y"` in xv6 but a.c, args_linux.c, b.c and `x  y` in bash, so neither the program nor the kernel expands * or removes quotes; the shell does.

Reading sh.c explained every difference that was observed: the string of special characters contains no * or $, the EXEC case calls exec with the name exactly as typed, which is why ls is not found outside the root folder, and cd is checked in main before fork, because only the parent process can change its own current folder. The experiments also showed two practical details of xv6 in a slow emulator: its console echoes typed characters immediately, even while a program is still printing, and it sends the escape sequences of keys such as the arrows to the terminal unchanged.

REFERENCES

1. The xv6 Shell from the Outside. Laboratory Work 4, Operating Systems course. Faculty of Computers, Informatics and Microelectronics, Technical University of Moldova, 2026.
2. COX, R., KAASHOEK, F., MORRIS, R. xv6: a simple, Unix-like teaching operating system. MIT, 2024 [online]. Available: https://pdos.csail.mit.edu/6.1810/2024/xv6/book-riscv-rev4.pdf
3. xv6 for RISC-V. Source code repository, MIT PDOS [online]. Available: https://github.com/mit-pdos/xv6-riscv
4. GNU Bash Reference Manual. Free Software Foundation [online]. Available: https://www.gnu.org/software/bash/manual/bash.html
5. execve(2). Linux manual page [online]. Available: https://man7.org/linux/man-pages/man2/execve.2.html